

SQL MASTERY — FROM THE BEGINNING

MySQL + MySQL Workbench | Beginner → Advanced Roadmap

1. What is SQL?

SQL means Structured Query Language. It is used to communicate with and manage relational databases.

Think of SQL as a language used to ask a database questions and perform operations on its data.

2. Database, Table, Row and Column

Database: A collection of related tables. **Table:** A structured collection of rows and columns. **Row:** One record.

Column: One attribute/type of information.

Company Database



- employees
- departments
- customers
- products
- orders

3. MySQL vs MySQL Workbench

MySQL is the database management system that stores and processes data. **MySQL Workbench** is a graphical tool used to connect to MySQL, write SQL queries, manage databases, and view results.

4. Create Your Practice Database

```
CREATE DATABASE sql_mastery;
USE sql_mastery;
```

5. Create Your First Table

```
CREATE TABLE employees (
  id INT,
  name VARCHAR(50),
  age INT,
  salary INT,
  department VARCHAR(50)
);
```

6. Insert Data

```
INSERT INTO employees
(id, name, age, salary, department)
VALUES
(1, 'Sanjay', 22, 50000, 'IT');
```

```
INSERT INTO employees
(id, name, age, salary, department)
VALUES
(2, 'Rahul', 23, 60000, 'HR');
```

```
INSERT INTO employees
(id, name, age, salary, department)
VALUES
(3, 'Priya', 24, 75000, 'IT');
```

7. SELECT — Read Data

```
SELECT * FROM employees;
```

```
SELECT name FROM employees;
```

```
SELECT name, salary  
FROM employees;
```

```
SELECT name, department  
FROM employees;
```

```
SELECT age  
FROM employees;
```

Mental model: SELECT = what columns? FROM = which table?

8. WHERE — Filter Rows

```
SELECT *  
FROM employees  
WHERE salary > 50000;
```

```
SELECT *  
FROM employees  
WHERE age = 23;
```

```
SELECT *  
FROM employees  
WHERE department = 'IT';
```

```
SELECT *  
FROM employees  
WHERE department <> 'IT';
```

Common comparison operators:

Operator	Meaning
=	equal
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal
<>	not equal

9. AND, OR, NOT

```
-- Both conditions must be true  
SELECT *  
FROM employees  
WHERE department = 'IT'  
AND salary > 50000;
```

```
-- At least one condition must be true  
SELECT *  
FROM employees  
WHERE department = 'IT'  
OR salary > 70000;
```

```
-- Reverse a condition  
SELECT *  
FROM employees  
WHERE NOT department = 'IT';
```

AND is strict: both conditions must be true. **OR** is flexible: at least one must be true. **NOT** reverses a condition.

10. IN and NOT IN

```
SELECT name  
FROM employees  
WHERE department IN ('IT', 'HR');
```

```
SELECT name  
FROM employees  
WHERE department NOT IN ('IT', 'HR');
```

IN means the value matches one of the listed values. NOT IN excludes all listed values.

11. BETWEEN

```
SELECT name
FROM employees
WHERE salary BETWEEN 50000 AND 70000;
```

```
SELECT name
FROM employees
WHERE salary NOT BETWEEN 50000 AND 70000;
```

Important: BETWEEN is inclusive. 50000 and 70000 are both included.

12. LIKE

```
-- Starts with S
SELECT name FROM employees WHERE name LIKE 'S%';
```

```
-- Ends with a
SELECT name FROM employees WHERE name LIKE '%a';
```

```
-- Contains 'an'
SELECT name FROM employees WHERE name LIKE '%an%';
```

```
-- Exactly one character represented by _
SELECT name FROM employees WHERE name LIKE 'S_njay';
```

Pattern	Meaning
'S%'	starts with S
'%a'	ends with a
'%an%'	contains an
'S_n'	one character between S and n

13. DISTINCT

```
SELECT department
FROM employees;
```

```
SELECT DISTINCT department
FROM employees;
```

DISTINCT removes duplicate result combinations from the selected columns.

14. ORDER BY

```
-- Ascending: smallest to largest
SELECT *
FROM employees
ORDER BY salary ASC;
```

```
-- Descending: largest to smallest
SELECT *
FROM employees
ORDER BY salary DESC;
```

```
-- Multiple columns
SELECT *
FROM employees
ORDER BY department ASC, salary DESC;
```

ASC = ascending. DESC = descending. With multiple columns, the first column has priority.

15. LIMIT

```
SELECT *
FROM employees
LIMIT 5;
```

```
-- Highest-paid employee
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

```
-- Top 3 salaries
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

```
-- Skip 2 rows and take 3
SELECT *
FROM employees
LIMIT 3 OFFSET 2;
```

16. Core SQL Thinking Pattern

```
SELECT columns
FROM table
WHERE conditions
ORDER BY column ASC/DESC
LIMIT number;
```

When solving a question, identify in this order: **What columns?** → **Which table?** → **Which rows?** → **How should they be sorted?** → **How many rows?**

17. Practice Problems

1. Display all employee information.
2. Display only employee names.
3. Find employees whose salary is greater than 60,000.
4. Find employees who work in IT.
5. Find employees who do not work in IT.
6. Find employees from IT or HR using IN.
7. Find employees whose salary is between 50,000 and 70,000.
8. Find employees whose name starts with S.
9. Find the highest-paid employee.
10. Find the top 2 highest-paid IT employees.
11. Display unique departments.
12. Display all employees sorted by salary from highest to lowest.
13. Find employees from IT or HR whose salary is not between 50,000 and 70,000.

18. Next Topics in the SQL Mastery Course

Aggregate Functions: COUNT, SUM, AVG, MIN, MAX
GROUP BY and HAVING
Primary Keys and Foreign Keys
Joins: INNER, LEFT, RIGHT, SELF, CROSS
Subqueries and correlated subqueries
CASE and conditional aggregation
UNION and UNION ALL
Constraints and database design
Normalization
Views, stored procedures and triggers
Transactions and ACID
Window functions: ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG
CTEs and recursive CTEs
Indexes and EXPLAIN
Query optimization
Real-world SQL interview problems

Golden Rules for Practice

- Always check the exact table name.
- Copy numeric conditions carefully: 6000 and 60000 are different values.
- Use single quotes for text values such as 'IT'.
- Remember LIKE patterns: 'S%' starts, '%a' ends, '%an%' contains.
- BETWEEN uses AND, not commas or parentheses.
- For top-N problems, usually combine ORDER BY ... DESC with LIMIT N.
- Do not memorize queries blindly. Translate the question into columns, table, conditions, sorting, and limit.